

Thread Synchronization Using Lock in C#

In this article, I am going to discuss **Thread Synchronization using Lock in C#** with Examples. Please read our previous article where we discussed [Thread Synchronization in C#](#) with Examples.

1. [Why do we need to Protect the Shared Resources in Multithreading in C#?](#)
2. [Accessing a Shared Resource in a Single-Threaded Environment in C#](#)
3. [Accessing a Shared Resource in a Multithreaded Environment in C#](#)
4. [How to Protect a Shared Resource in a Multithread Environment from Concurrent Access in C#?](#)
5. [What is the Lock Statement in C#?](#)
6. [How is the Lock Statement internally work in C#?](#)
7. [Protecting Shared Variable using Lock Statement in C# with Example](#)

Why do we need to Protect the Shared Resources in Multithreading in C#?

In a multithreading application, it is very important for us to handle multiple threads for executing critical section code or you can say shared resources. For example, if we have a shared resource, and multiple threads want to access the shared resource, then we need to protect the shared resource from concurrent access otherwise we will get some inconsistent behavior or output.

In C#, we can use Lock and Monitor to provide thread safety in a multithreaded application. Both lock and monitor provide a mechanism that ensures that only one thread is executing the critical section code at any given point in time to avoid any functional breaking of code or to avoid inconsistent behavior or output.

In this article, I am going to discuss how to protect the shared resource in a multithread environment using the lock and in the next article, I am going to discuss how to do the same thing using the [Monitor in C#](#).

Accessing a Shared Resource in a Single-Threaded Environment in C#:

Before understanding how to use the lock to protect the shared resource in a multithread environment in C#, let us first understand the problem if we are not protecting the shared resource in a multithread environment. In the following example, we have a shared resource i.e. DisplayMessage() method and we call that method three times from the Main method as shown below.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            DisplayMessage();
            DisplayMessage();
            DisplayMessage();
            Console.Read();
        }
        static void DisplayMessage()
```

```

    {
        Console.Write("[Welcome to the ");
        Thread.Sleep(1000);
        Console.WriteLine("world of dotnet!"]);
    }
}

```

Output:

```

[Welcome to the world of dotnet]
[Welcome to the world of dotnet]
[Welcome to the world of dotnet]

```

As the above program is a single-threaded program, so we got the output as expected. Here, the shared resource is the DisplayMessage method which is going to be executed three times sequentially, and hence we got the correct output. Now, let us proceed and see what happens if we access the shared resources in a multithreaded environment.

Accessing a Shared Resource in a Multithreaded Environment in C#:

In the following example, we have created three different threads and then invoke the same DisplayMessage() method using all these three different threads. Here, DisplayMessage() method is the shared resource and this shared resource is simultaneously or concurrently invoked by three different threads. Here, we are not protecting the shared resource, and all three threads accessing the shared resource which results in inconsistent output.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread t1 = new Thread(DisplayMessage);
            Thread t2 = new Thread(DisplayMessage);
            Thread t3 = new Thread(DisplayMessage);
            t1.Start();
            t2.Start();
            t3.Start();
            Console.Read();
        }
        static void DisplayMessage()
        {
            Console.Write("[Welcome to the ");
            Thread.Sleep(1000);
            Console.WriteLine("world of dotnet!"]);
        }
    }
}

```

```
}  
}
```

Output:

```
[Welcome to the [Welcome to the [Welcome to the world of dotnet!]  
world of dotnet!]  
world of dotnet!]
```

As you can see, here we are not getting the output as expected. So, the point that you need to keep in mind is that if the shared resource is not protected in a multithreaded environment from concurrent access, then the output or the behavior of the application becomes inconsistent.

How to Protect a Shared Resource in a Multithread Environment from Concurrent Access in C#?

We can protect the shared resources in a multithread environment from concurrent access by using the concept Monitor and Locking in C#. Let us see how to protect the shared resource using the lock statement in C# and see the output. In the following example, we have created one readonly object i.e LockDisplayMethod, and then we created a block using the lock keyword. To the lock keyword, we pass the LockDisplayMethod object, and the section or block or particular resource that we want to protect should be placed inside the lock block which is shown in the below example.

```
using System.Threading;  
using System;  
namespace ThreadingDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Thread t1 = new Thread(DisplayMessage);  
            Thread t2 = new Thread(DisplayMessage);  
            Thread t3 = new Thread(DisplayMessage);  
            t1.Start();  
            t2.Start();  
            t3.Start();  
            Console.Read();  
        }  
        private static readonly object LockDisplayMethod = new object();  
        static void DisplayMessage()  
        {  
            lock(LockDisplayMethod)  
            {  
                Console.Write("[Welcome to the ");  
                Thread.Sleep(1000);  
                Console.WriteLine("world of dotnet!]");  
            }  
        }  
    }  
}
```

```
}  
    }  
}
```

Now run the application and see the output as expected as shown below. As you can see in the below output, we are getting the result as expected and this is because we are now protecting the shared resource using the lock statement which will ensure that only one thread will be able to access the critical section at any given point in time.

```
[Welcome to the world of dotnet!]  
[Welcome to the world of dotnet!]  
[Welcome to the world of dotnet!]
```

What is the lock statement in C#?

According to [Microsoft](#), the lock statement acquires the mutual-exclusion lock for a given object, executes a statement block, and then releases the lock. While a lock is held, the thread that holds the lock can again acquire and release the lock. Any other thread is blocked from acquiring the lock and waits until the lock is released.

Note: When you want to synchronize thread access to a shared resource, you should lock the shared resource on a dedicated object instance (for example, `private readonly object _lockObject = new object();` or `private static readonly object _lockObject = new object();`). Avoid using the same lock object instance for different shared resources, as it might result in a deadlock.

How is the lock Statement internally work in C#?

The lock statement in C# was internally converted to a try-finally block when we compiled the code. The compiled code of the lock statement will look like the below. You can see, it is internally using the Monitor class Enter and Exit method. In our next article, we will discuss the [Monitor Class Enter and Exit Methods](#) in detail, for now, to understand, what we can say is, it acquires an exclusive lock within the try block by invoking the Monitor class Enter method and it releases the exclusive lock within the finally block by calling the Monitor class Exit method.

```
bool lockWasTaken = false;
var temp = new object;
try
{
    Monitor.Enter(temp, ref lockWasTaken);
    // body
}
finally
{
    if (lockWasTaken)
    {
        Monitor.Exit(temp);
    }
}
```

You can also verify the IL Code by using the ILDASM tool. Now, if you open the application using the ILDASAM tool and if you verify the DisplayMessage method IL Code, then you can see the try and finally blocks along with the Monitor class Enter and Exit methods as shown in the below image.

```

.method private hidebysig static void DisplayMessage() cil managed
{
    // Code size          67 (0x43)
    .maxstack 2
    .locals init ([0] object U_0,
                  [1] bool U_1)
    IL_0000:  nop
    IL_0001:  ldsfld      object ThreadingDemo.Program::LockDisplayMethod
    IL_0006:  stloc.0
    IL_0007:  ldc.i4.0
    IL_0008:  stloc.1
    .try
    {
        IL_0009:  ldloc.0
        IL_000a:  ldloca.s     U_1
        IL_000c:  call         void [mscorlib]System.Threading.Monitor::Enter(object,
                                                                bool&)

        IL_0011:  nop
        IL_0012:  nop
        IL_0013:  ldstr        "[Welcome to the "
        IL_0018:  call         void [mscorlib]System.Console::Write(string)
        IL_001d:  nop
        IL_001e:  ldc.i4       0x3e8
        IL_0023:  call         void [mscorlib]System.Threading.Thread::Sleep(int32)
        IL_0028:  nop
        IL_0029:  ldstr        "world of dotnet!]"
        IL_002e:  call         void [mscorlib]System.Console::WriteLine(string)
        IL_0033:  nop
        IL_0034:  nop
        IL_0035:  leave.s      IL_0042
    } // end .try
    finally
    {
        IL_0037:  ldloc.1
        IL_0038:  brfalse.s   IL_0041
        IL_003a:  ldloc.0
        IL_003b:  call         void [mscorlib]System.Threading.Monitor::Exit(object)
        IL_0040:  nop
        IL_0041:  endfinally
    } // end handler
    IL_0042:  ret
} // end of method Program::DisplayMessage

```

Protecting Shared Variable using Lock Statement in C# with Examples:

The section or block or particular resource that you want to protect should be placed inside the lock block. Let us understand this with an example. In the below example, we are only protecting the shared Count variable from concurrent access.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static int Count = 0;

```

```

static void Main(string[] args)
{
    Thread t1 = new Thread(IncrementCount);
    Thread t2 = new Thread(IncrementCount);
    Thread t3 = new Thread(IncrementCount);
    t1.Start();
    t2.Start();
    t3.Start();
    //Wait for all three threads to complete their execution
    t1.Join();
    t2.Join();
    t3.Join();
    Console.WriteLine(Count);
    Console.Read();
}
private static readonly object LockCount = new object();
static void IncrementCount()
{
    for (int i = 1; i <= 1000000; i++)
    {
        //Only protecting the shared Count variable
        lock (LockCount)
        {
            Count++;
        }
    }
}
}
}

```

When you run the above program, it will give you the output as expected as 3000000. Now, let us see what happens if we are not protecting our shared variable Count. In the below example, we are not protecting the Count shared variable and hence all three threads simultaneously access the variable and try to increment the value, and hence we will get some unexpected output.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static int Count = 0;
        static void Main(string[] args)
        {
            Thread t1 = new Thread(IncrementCount);
            Thread t2 = new Thread(IncrementCount);
            Thread t3 = new Thread(IncrementCount);
            t1.Start();
            t2.Start();
            t3.Start();
            //Wait for all three threads to complete their execution

```

```

        t1.Join();
        t2.Join();
        t3.Join();
        Console.WriteLine(Count);
        Console.Read();
    }

    static void IncrementCount()
    {
        for (int i = 1; i <= 1000000; i++)
        {
            Count++;
        }
    }
}

```

Every time, you run the application, you will get a different output. So, it is important for us to protect our shared resources in a multithreaded application, or else we will not get the expected output.

Here, in this article, I try to explain **Thread Synchronization using Lock in C#** with Examples. In the next article, I am going to discuss [How to Protect Shared Resources in a Multithread Environment using Monitor Class in C#](#) with Examples. I hope you enjoy this Thread Synchronization using Lock in C# with Examples article.